

Research Document

Project Title: Gamified App For Student

Author: Shuai Jiang

Date: 12/10/2025

1. Introduction.....	4
1.1 Problem Statement.....	4
1.2 Project Vision & Goals.....	4
2. Gamification.....	4
2.1 How It Works.....	4
2.2 How To Implement.....	5
2.3 Potential Challenges and Issues.....	5
3. Market and Competitive Analysis.....	7
3.1 Analysis: Habitica.....	7
3.1.1 Core Concept.....	7
3.1.2 Gamification Mechanics.....	7
3.1.3 Something Can Learn From This App.....	8
A. Valuable Concepts I Can Borrow.....	8
B. Elements I Should Avoid.....	8
C. Concepts I Can Adapt.....	8
3.2 Analysis: Forest.....	9
3.2.1 Core Concept.....	9
3.2.2 Gamification Mechanics.....	9
3.2.3 Something Can Learn From This App.....	10
A. Valuable Concepts I Can Borrow.....	10
B. Elements I Should Avoid.....	10
C. Concepts I Can Adapt.....	10
3.3 Analysis: Headspace.....	11
3.3.1 Core Concept.....	11
3.3.2 Gamification Mechanics.....	11
3.3.3 Something Can Learn From This App.....	12
A. Valuable Concepts I Can Borrow.....	12
B. Elements I Should Avoid.....	12
C. Concepts I Can Adapt.....	12
3.4 Analysis: Duolingo.....	13
3.4.1 Core Concept.....	13
3.4.2 Gamification Mechanics.....	13
3.4.3 Something Can Learn From This App.....	14
A. Valuable Concepts I Can Borrow.....	14
B. Elements I Should Avoid.....	14
C. Concepts I Can Adapt.....	14
3.5 Comparative Analysis of Other Apps.....	14
3.6 Project's Position.....	15
4. Technology Investigation.....	16
4.1 Mobile Platform Investigation.....	16
4.1.1 Comparing Android vs. iOS Platforms.....	16

4.1.2 Comparing Development Stacks (Kotlin vs. Flutter).....	16
4.1.3 Decision: Native Android with Kotlin.....	17
4.2 Mobile Platform.....	18
4.3 Development Stack.....	18
Architecture: Decoupled (Client-Server) vs. Monolithic.....	18
Backend Framework: Django vs. Flask.....	18
Frontend Platform: Native Android (Kotlin) vs. Cross-Platform (Flutter).....	19
Production Database: PostgreSQL vs. MySQL/NoSQL.....	19
API and Testing (DRF and Postman).....	20
4.4 Hosting.....	20
5.Conclusion.....	20
Reference.....	21

1. Introduction

1.1 Problem Statement

Higher education students today face a unique set of challenges. They often struggle with disorganized tasks from various sources, difficulty in maintaining motivation for self-directed study, and a sense of social isolation on a large campus. Balancing these academic pressures with personal well-being is a challenge.

1.2 Project Vision & Goals

This project is a mobile app designed to help students manage their studies and personal growth in a more engaging way. To keep students motivated, the app uses several game-like features. By completing personal goals and real-world activities, students can earn Points, unlock achievement Badges, and compete on staged Leaderboards. What makes this app different is its focus on the real world. Instead of encouraging more online chatting, its key goal is to provide students with a tool to practice and improve their face-to-face social skills.

2. Gamification

2.1 How It Works

Based on my research, gamification is the process of applying game-like elements to non-game contexts to increase user engagement and motivation. For my project, this is the core strategy to solve the problem of student motivation decline.

The system works by creating a positive feedback loop. When a student completes a real-world task that is beneficial to them, the app provides an immediate virtual reward. This psychological reward helps reinforce the positive behavior, making the student more likely to repeat it. The key elements I plan to use are:

Points: This is the most direct form of feedback and the core currency of the app's economy. Completing almost any positive action, such as finishing a task or a successful focus session, will grant the user experience points (XP). These points serve a dual purpose: they act as a clear measure of the user's effort, contributing to their level progression, and they also function as "credits" that can be spent in the in-app shop, giving every point earned a tangible value.

Badges: These are virtual trophies awarded for achieving specific, meaningful milestones. Badges serve as long-term goals and a way for users to display their accomplishments. To add more depth, I plan to implement a tiered badge system (e.g., bronze, silver, gold). For example, a student might earn a bronze "Focus Master" badge after 10 hours of focused study, a silver one after 50 hours, and a gold one after 150 hours. This tiered structure provides a continuous sense of progression and encourages long-term commitment.

Leaderboards: To add a social and competitive element, the app will use a staged or league-based leaderboard system. Instead of a single, global ranking where new users might feel discouraged by the huge gap with top players, this system places users into small, competitive groups (e.g., 30-50 people) with others at a similar activity level. The competition runs on a weekly cycle. At the end of the week, the top performers in each group are promoted to a higher league, while the lowest performers may be demoted. This structure ensures that every user, regardless of their overall rank, has an achievable and relevant goal each week—either fighting for promotion or striving to avoid demotion—which keeps the competition consistently fair and engaging.

2.2 How To Implement

To ensure the rules are consistent and secure, the core logic for the gamification engine will be implemented on the backend (server-side) using Python and Django.

When the user completes an action in the mobile app (the frontend), the app will send a request to my backend API (e.g., POST /api/tasks/complete).

The backend will then verify this action, calculate the appropriate points and check if any badges have been earned.

Finally, the backend will update the user's data in the database and send a confirmation back to the mobile app, which will then display a notification to the user (e.g., "You earned 20 points!").

By handling this logic on the server, I can easily adjust the rules (like how many points a task is worth) without needing to update the mobile app itself, and it ensures a fair and consistent experience for all users.

2.3 Potential Challenges and Issues

While gamification is a powerful tool, my research indicates several potential issues that I must carefully manage during development:

- **The Overjustification Effect:** This is a key risk. By providing external rewards (Points, Badges) for activities that should be intrinsically motivating (like learning or socializing), there's a danger that users might start to feel they are

only doing these activities for the points. If the rewards are removed, their intrinsic motivation might be lower than when they started.

Strategy: I will keep the rewards supportive rather than transactional. The UI/UX, inspired by Headspace , will focus on "growth" and "achievement" rather than just "earning," to protect that intrinsic motivation.

- **Unbalanced Economy:** It will be a challenge to balance the points earned from different activities (e.g., finishing a task vs. a 1-hour focus session vs. a social interaction). If one activity is significantly more "profitable" than others, users might focus only on that single activity, which defeats the app's purpose of promoting a balanced student life.

Strategy: The backend-driven rules system is crucial. It will allow me to easily tune and adjust the point values for each activity during testing, to ensure all core behaviors are encouraged relatively equally.

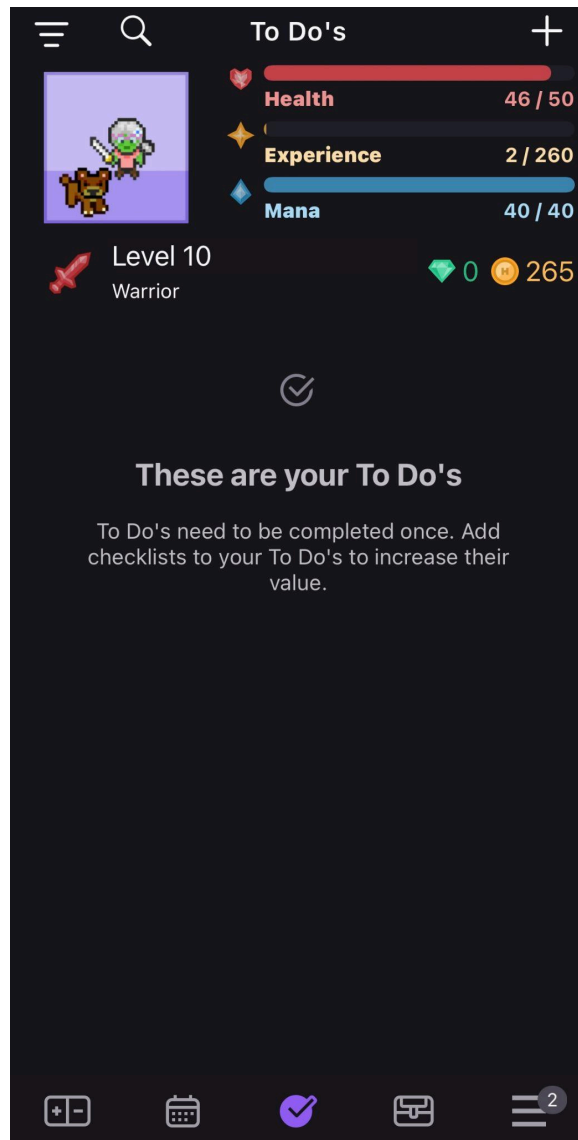
- **Leaderboard Demotivation:** While the Leaderboard is an **Essential** feature, competition can still be stressful for some students. Users who consistently find themselves at the bottom of their league might feel demotivated and abandon the app.

Strategy: My primary strategy to combat this is the **design of the leaderboard itself**.

1. By implementing the **staged-league system** (inspired by Duolingo), users are not competing against everyone, but only within a small group of similarly active peers. This makes the competition feel fair and achievable .
2. Secondly, while the leaderboard provides *competitive* motivation, the app will equally emphasize **personal progression** through **Badges** and **Levels** . This gives users a parallel path to feel a sense of achievement, even if they aren't at the top of their league that week.
3. Finally, by adapting concepts like the "Streak Freeze" from Duolingo for our "Credits" shop , we can help reduce the *anxiety* associated with performance, which indirectly affects the leaderboard experience.

3. Market and Competitive Analysis

3.1 Analysis: Habitica



3.1.1 Core Concept

After using it for a week, I found that Habitica is an innovative productivity tool. My understanding is that it changes a user's real-life tasks (including habits, daily to-do, and normal tasks) into a complete role-playing game (RPG). I saw that users get experience and gold by completing tasks in real life, which upgrades their virtual character in the game. I believe this method makes the process of completing boring tasks full of fun.

3.1.2 Gamification Mechanics

Some important features I identified are: Experience Points (XP) & Levels; Gold & Reward Shop; Health Points (Health) & Punishment; Character Role-Playing

Elements: Equipment, Pets, and Mounts; and a Social System where users can form "Guilds".

3.1.3 Something Can Learn From This App

A. Valuable Concepts I Can Borrow

Punishment Mechanism: I think Habitica's "lose Health" punishment is very effective because it makes the user's promised tasks feel more important. For my own app, I can learn from this idea and use a more gentle consequence method. For example, if a user fails their "Focus Mode" challenge, they might lose a small number of points. I feel this is stronger than just getting "no reward."

Internal Economy System: I also found that Habitica's Gold and Shop system gives a second value to the user's effort, besides just upgrading. I believe I can learn from this point. In my app, I can let students use their earned points not only to level up, but also to exchange for personalized rewards (like new app themes or avatar looks) in a simple "shop". In my opinion, this will make earning points have more purpose.

B. Elements I Should Avoid

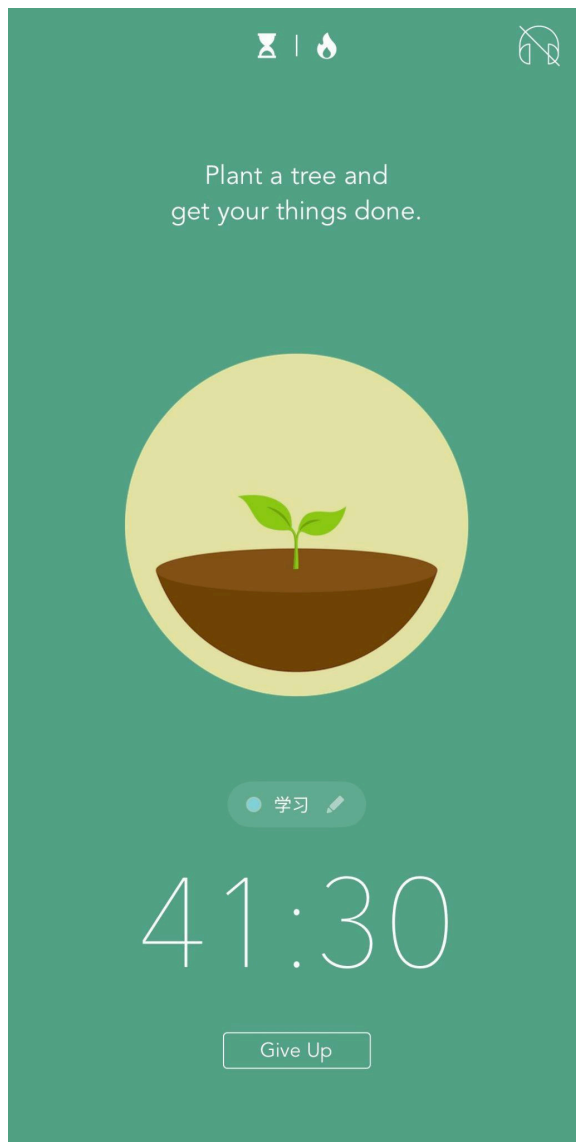
Overly Complex RPG System: For me, this is the most important point. I found that Habitica's classes, equipment, and pet systems are very attractive to game players, but I think they might distract students' attention. My opinion is that users may spend a lot of their study time to research these game systems, which is opposite to my app's main goal of "improving study efficiency." For my project, I want to keep it simple, to make sure the gamification is for motivation, not the final goal itself.

Unique UI Style: I can accept Habitica's pixel art style, but I agree it is not for everyone. For an app that I want to build to help students focus and grow, I believe a more modern, simple, and calm interface is a better strategic choice.

C. Concepts I Can Adapt

Changing "Guilds" into Offline "Study Groups": I saw that Habitica's "Guilds" is a pure online social function. I think I can learn from its "team work" idea and adapt it to serve my "offline social" core goal. For example, in my app, users could form "Study Groups." When group members use the Bluetooth function to confirm they had an offline meeting, the whole group would get extra rewards. I think this will effectively motivate real study communication.

3.2 Analysis: Forest



3.2.1 Core Concept

After using Forest, I found its core concept is to use a simple and beautiful metaphor—growing a tree—to help users stay focused and put down their phones. It's essentially a gamified timer that gives users a positive visual reward for their period of concentration.

3.2.2 Gamification Mechanics

Forest's gamification is very direct and effective. The main mechanics I identified are:

Positive Reward (Positive Reinforcement): Successfully completing a focus session results in a new tree being added to your personal forest.

Negative Consequence (Punishment): If you leave the app during a focus session, the growing tree will wither and die. This dead tree permanently stays in your forest as a reminder of the failure.

Accumulation & Visualization: Over time, successful sessions build a dense, personalized forest, visually representing all the time the user has dedicated to focusing.

In-App Currency: Successful sessions also earn coins, which can be used to unlock new tree species.

3.2.3 Something Can Learn From This App

A. Valuable Concepts I Can Borrow

The Power of Consequences: The idea of a tree withering is a very powerful psychological penalty. It's much more effective than simply not getting a reward. This reinforces my decision to include a penalty for failing the "Focus Mode" in my app, such as losing a small number of points.

Visualizing Effort: The concept of building a forest is a great way to visualize accumulated effort. I can adapt this idea not by building a forest, but perhaps by creating a "virtual trophy room" or a customizable profile page where unlocked badges are displayed, creating a visual record of achievement.

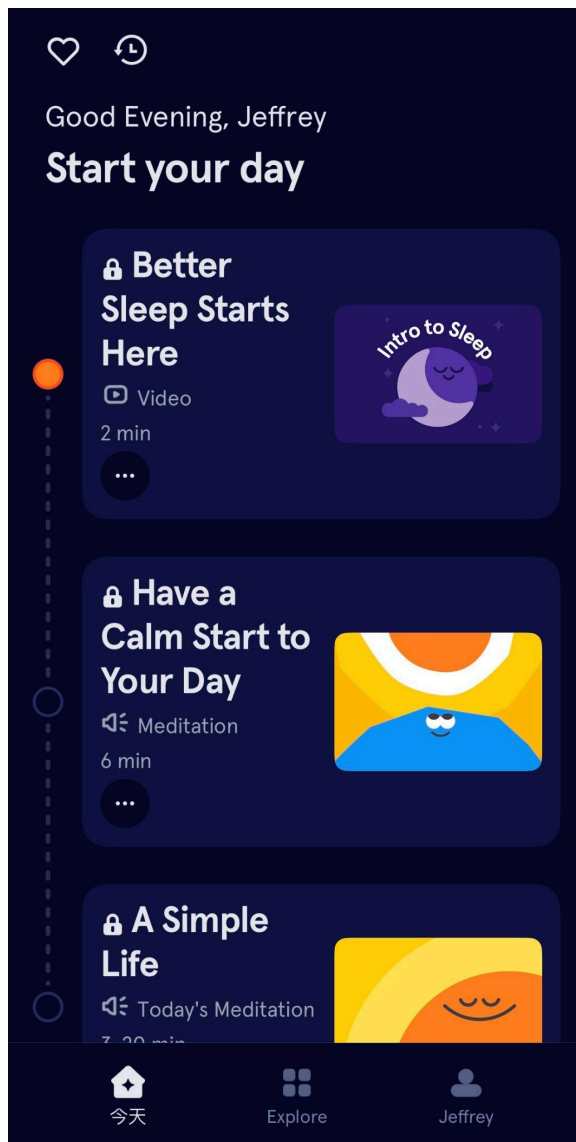
B. Elements I Should Avoid

Single-Purpose Focus: Forest is an excellent app, but it does one thing: focus timing. My project is a multi-functional companion for students. Therefore, I cannot make the entire user experience revolve around a single feature. The Focus Mode needs to be a well-integrated part of a larger ecosystem of features.

C. Concepts I Can Adapt

Adapting "Group Focus": Forest allows multiple users to plant a tree together online, and if one person gives up, everyone's tree dies. I can adapt this concept for my "offline social" goal. For example, when members of a "Study Group" in my app confirm they are meeting in person via Bluetooth, they could start a "Group Focus Session." If the group successfully completes the session, everyone receives a significant point bonus, which would strongly encourage productive, real-world study groups.

3.3 Analysis: Headspace



3.3.1 Core Concept

After trying Headspace, my analysis is that it is a subscription-based service focused on mental well-being. It offers a large library of guided meditations, sleep aids, and mindfulness exercises. I see that it positions itself as a "gym membership for the mind," aiming to make mindfulness an accessible daily habit.

3.3.2 Gamification Mechanics

I found that Headspace's use of gamification is very subtle and supportive, rather than competitive. The main mechanics are:

Daily Streaks: It tracks and rewards the number of consecutive days a user completes a meditation session.

Progress Tracking: It visually displays the user's total accumulated meditation time.

Positive Reinforcement: The entire app is filled with encouraging language and calming, rewarding animations after a session is completed.

3.3.3 Something Can Learn From This App

A. Valuable Concepts I Can Borrow

Intuitive and Calming UI/UX: For me, the biggest lesson from Headspace is the importance of a clean, calm, and positive user interface. The app uses soft colors, friendly illustrations, and a simple layout that feels supportive, not stressful. As you pointed out, users don't need to specifically learn how to use it. This principle of a **"low learning curve"** is something I must apply to my own design, to ensure my app helps reduce student stress, not add to it.

Structured Daily Engagement: Headspace offers a "Meditation of the Day" which encourages users to open the app daily. I can borrow this idea for my project by creating a "Social Challenge of the Week" or a "Daily Wellness Tip" to provide a simple, recurring reason for users to check in.

B. Elements I Should Avoid

Subscription Model: Headspace's main business is selling access to content. My app, however, is a tool, not a content library. I will avoid placing any of my core productivity or social features behind a paywall.

Passive Experience: I noticed the primary user action in Headspace is passive (listening). My app's goal is to encourage active behaviors like completing tasks and meeting people. Therefore, I need to ensure my features are interactive and prompt the user to do something.

C. Concepts I Can Adapt

Adapting "Mindful Moment" Notifications: Headspace can send gentle notifications to remind users to take a break. I can adapt this into a much simpler, optional **"Wellness Check-in"** feature. This could be a single daily notification asking, "How are you feeling today?" which users can quickly respond to for a small point reward. I think this would be a lightweight and effective way to address the "Mental Health" aspect of my project.

3.4 Analysis: Duolingo



3.4.1 Core Concept

My analysis of Duolingo is that it is a language-learning platform that has successfully turned the entire, often difficult, process of learning a language into a fun and accessible game. I found that it breaks down complex language skills into small, bite-sized lessons and uses a powerful gamification engine to motivate users to practice every day.

3.4.2 Gamification Mechanics

Duolingo is a masterclass in gamification. The key mechanics I have focused on are:

Leagues (Leaderboards): The core competitive feature, placing users in weekly competitions against a small group of peers.

Streaks: Tracks the number of consecutive days a user has met their daily goal.

Experience Points (XP): Earned for completing lessons, and serves as the basis for the leaderboard ranking.

In-App Currency (Gems/Lingots): A virtual currency earned through use, which can be spent in the shop.

Achievements (Badges): A set of badges awarded for hitting various milestones (e.g., total XP earned, number of perfect lessons).

3.4.3 Something Can Learn From This App

A. Valuable Concepts I Can Borrow

The League System Model: Duolingo's league-based leaderboard is the **direct model** for my app's staged leaderboard. The way it places users in small, weekly competitive groups (e.g., 30 people) with clear promotion and demotion zones is something I plan to implement directly. I believe this system is brilliant because it makes the competition feel fair and consistently motivating, which will be very effective for students.

The Streak & In-App Economy: The "Daily Streak" is a proven mechanic for building habits, which I will definitely include in my app. More importantly, Duolingo's "**Streak Freeze**"—an item that can be bought with Gems to save a streak—is a perfect example for my in-app "Credits" store. It shows how to offer items that provide real value and reduce user anxiety, rather than just being cosmetic. This directly relates to my supervisor's suggestion to include a way for users to "buy things".

B. Elements I Should Avoid

Aggressive Notifications: I know from experience that Duolingo is famous for its very persistent notifications. For my app, which is aimed at helping students reduce stress, I will adopt a calmer and more respectful notification strategy. The notifications should be helpful reminders, not a source of pressure.

Linear Learning Path: Duolingo's lesson structure is very linear and prescribed; you must complete one level to unlock the next. My app is not a course, but a toolbox. Therefore, the user experience must be flexible, allowing students to use the features they need (like the To-Do List or Focus Mode) in any order they choose.

C. Concepts I Can Adapt

The main adaptation from Duolingo is taking its core motivational engine (Leagues and Streaks), which is designed for a single, linear goal (learning a language), and applying it to the more diverse and user-driven context of student life. The challenge will be to balance the point rewards from different activities (studying, socializing, etc.) to create a fair and engaging overall experience.

3.5 Comparative Analysis of Other Apps

This table summarizes the key insights learned from other relevant applications.

App Name	Relevant Feature in Our Project	Key Gamification Method	Strengths to Borrow	Weaknesses / Differences
Habitica	Gamification Engine & Personal Task Management	A deep RPG system with XP, Health points (punishment), Gold (in-app currency), Equipment, mounts, pets, and social Guilds.	The "Consequence Mechanism" (losing health) is powerful. The "In-App Economy" (spending currency) gives points real value.	The RPG system is overly complex and can be a distraction. The social features are online-only. The pixel art style is niche.
Forest	Focus Mode	Successfully focusing grows a tree; failing kills the tree. The accumulated trees form a visual forest.	The "group focus" concept is a perfect inspiration for our offline "Study Groups" .	Forest is almost entirely a focus timer. However, the application we are developing is a multifunctional student assistant app.
Headspace	UI/UX Design & Mental Well-being Module	It primarily uses daily streaks and a calm, encouraging UI/UX to build a habit.	The simple, calm and intuitive interface is deeply loved by users, and it also has a simple operation logic .	Headspace is about passive content consumption (listening) . Our app's core is to encourage active user behaviors (completing tasks, socializing) .
Duolingo	Gamification Engine (Leaderboards & Economy)	Its core loop is driven by a highly competitive weekly league system, daily streaks, and virtual currency (gems) used to purchase items such as "Streak Freeze".	The league-based leaderboard is the perfect model for our staged leaderboard system. The "Streak Freeze" is an excellent idea for our in-app "Credits" store.	The learning path is highly structured , which may make people feel monotonous and repetitive, and a bit boring . Our application needs to become a flexible toolbox. Its notification strategy might be overly aggressive.

3.6 Project's Position

After analyzing these successful applications, we can find that they are usually designed as universal tools for a broad audience. They lack specific attention to the lives of students.

"Gamified App For Students" precisely targets this gap. Its unique value is based on two core principles:

Specifically for students: The functions of this application are tailored to the daily schedules and challenges faced by students.

Encouraging offline social interaction: The key feature of this project lies in its innovative Bluetooth-based function, which rewards face-to-face interactions.

4. Technology Investigation

4.1 Mobile Platform Investigation

4.1.1 Comparing Android vs. iOS Platforms

In choosing a development platform, I first compared the two main ecosystems: Android and iOS.

Android (Google):

- **Advantages:** This is an open ecosystem. Developers can freely test on any device, and the process for publishing to the Google Play Store is relatively faster and has fewer restrictions. It has a very high global market share, especially in Europe and Asia, which provides a broader potential user base for the application.
- **Disadvantages:** "Fragmentation" is the biggest challenge. Due to the large number of different manufacturers (Samsung, Google, Huawei, etc.), screen sizes, and operating system versions, ensuring the app runs well on all devices requires more testing and adaptation effort.

iOS (Apple):

- **Advantages:** This is a closed, highly unified ecosystem. Developers only need to develop for a few iPhone and iPad models, meaning the device fragmentation problem is almost non-existent. This makes it much easier to guarantee a high-quality and consistent user experience.
- **Disadvantages:** The development and publishing process is stricter. Development must be done on macOS (requiring an Apple computer), and apps are subject to a strict and lengthy review process by Apple when submitting to the App Store.

4.1.2 Comparing Development Stacks (Kotlin vs. Flutter)

After deciding on a mobile-first strategy, I compared two mainstream development technologies: Native (Kotlin) and Cross-Platform (Flutter).

Comparison Dimension	Native (Kotlin for Android)	Cross-Platform (Flutter with Dart)
Performance	Code is compiled directly to native machine code, fully leveraging device performance.	Flutter bypasses native components and draws UI directly on a canvas, with performance close to native.
Development Efficiency	Development services one platform at a time (Android).	A single codebase can be compiled for both Android and iOS apps, theoretically saving 50% of the time.
UI Consistency	The app's look and feel will perfectly match the system habits of Android users.	Because Flutter has its own rendering engine, the app's appearance and interaction are almost identical on Android and iOS.
Device Feature Access	Can seamlessly and reliably call all native APIs (like Bluetooth, GPS, camera).	Needs to use "channels" to call native APIs. For complex features like Bluetooth, it may require extra configuration and reliability is slightly lower than native.
Learning Curve	This is a personal advantage, as I have prior experience with Kotlin.	Requires learning a new language (Dart) and a completely new responsive UI framework (Flutter) simultaneously.

4.1.3 Decision: Native Android with Kotlin

Based on this analysis, I have chosen Native Android with Kotlin for the initial development.

The project requirement allows for developing on "Android and/or iOS," and I believe concentrating my efforts on one platform will allow me to build a more feature-complete and high-quality application, rather than dividing my time between two.

While Flutter's cross-platform efficiency is attractive, the requirement to learn a new language and framework (Dart) would introduce significant risk and overhead. My existing experience with Kotlin further reinforces this decision, as it removes the learning curve and allows me to focus immediately on building features. Furthermore, our project's key innovative feature (UC-RW-03) relies heavily on Bluetooth, which is a native device feature. Choosing native Kotlin development ensures I will have the most direct and reliable access to the Bluetooth module, minimizing technical risks in the project's most complex area.

Finally, the project's decoupled backend architecture ensures that developing an iOS client in the future would be a straightforward process, as the core logic will already be in place.

4.2 Mobile Platform

For this project, I will be developing for the Android platform as the primary target.

The project requirement allows for developing on "Android and/or iOS," and I've chosen to focus on a native Android app because I believe concentrating my efforts on one platform will allow me to build a more feature-complete and high-quality application, rather than dividing my time between two.

Android was selected specifically due to several factors, including the open-nature of its development ecosystem, the wide availability of devices for testing, and its significant global market share. Furthermore, the project's decoupled backend architecture ensures that developing an iOS client in the future would be a straightforward process, as the core logic will already be in place.

4.3 Development Stack

Here is a brief introduction and justification for the specific tools I chose, including a comparison against other viable alternatives.

Architecture: Decoupled (Client-Server) vs. Monolithic

I chose a **decoupled (client-server) architecture**, where the backend and frontend are separate projects. The alternative would be a monolithic architecture, where the frontend and backend are tightly combined into a single application.

While a monolithic approach can be simpler for basic websites, it is not well-suited for a project that needs to support a native mobile application. The decoupled architecture allows me to build a flexible **REST API** on the backend, which can then serve any client—be it my Android app, a future iOS app, or a website—making it a much more modern and scalable choice.

Backend Framework: Django vs. Flask

For the backend, I will be using Python with the **Django framework**. Another popular option in the Python world is Flask.

Flask is an excellent "micro-framework" that is very lightweight and flexible. However, it requires adding many separate libraries for features like user authentication, database management, and an admin interface. For my project, **Django's "batteries-included"** approach is a significant advantage. Its built-in user login system and admin pages are powerful features that I would have to build myself in Flask. By choosing Django, I can save a great deal of time on these standard features and focus my energy on the unique and interesting parts of my application, such as the gamification logic.

Frontend Platform: Native Android (Kotlin) vs. Cross-Platform (Flutter)

For the mobile application, I will be building a native Android app using Kotlin. I considered using a cross-platform framework like Flutter, which could build for both Android and iOS from a single codebase.

However, the cross-platform approach would require me to learn a new language (Dart) and a completely new framework. By choosing native Android with Kotlin, I can focus on mastering one ecosystem. This allows for the best possible performance, direct access to all native device features (which may be important for the Bluetooth module), and a more polished user experience. Since the project requirement allows for "Android and/or iOS", I believe delivering one high-quality native application is a better strategy than potentially struggling with two platforms at once.

Production Database: PostgreSQL vs. MySQL/NoSQL

I am planning a two-stage approach for the project's database.

For **initial development**, I will use **SQLite**. This is because SQLite is the default database for Django, it requires zero setup, and it's stored in a single file within my project. This makes it incredibly simple and fast for me to get started, prototype features, and test the application on my local machine without worrying about server configuration.

For the final, **production-ready version** of the project, I plan to migrate to **PostgreSQL**. The main reason for this migration is to handle **concurrency**. SQLite is a file-based database and does not handle multiple simultaneous connections or write operations well. A live application with many students trying to complete tasks, use the focus mode, or log social interactions at the same time would quickly run into performance bottlenecks and data locking issues with SQLite. PostgreSQL, on the other hand, is a powerful client-server database specifically designed to handle many concurrent connections robustly and efficiently, which is essential for any multi-user application.

Additionally, PostgreSQL offers better **scalability** for a growing user base and a stricter, richer set of data types, which helps ensure **data integrity** as the application

becomes more complex.

This two-stage approach allows for rapid initial development with SQLite, while also planning for the robustness, performance, and concurrency required for a real-world application with PostgreSQL.

API and Testing (DRF and Postman)

To enable communication between the back-end and front-end, I will use the Django REST framework (DRF) to create a simple REST API. At the same time, I will use tools like Postman to test the API during its construction. This way, I can ensure that the back-end logic can run independently before connecting it to the mobile application.

Version Control (Git and GitHub): I will use Git and GitHub to save and manage all my code and documents. The specific operations will be carried out as needed.

4.4 Hosting

For the final project demonstration, the Django backend will need to be deployed to a live server. My research considered two main types of cloud services: Infrastructure-as-a-Service (IaaS) and Platform-as-a-Service (PaaS).

IaaS providers, such as **Amazon Web Services (AWS)** or **Google Cloud**, offer maximum flexibility by providing raw server infrastructure. However, this approach requires significant expertise in server administration, including setting up the operating system, web server, database, and security configurations. The learning curve is very steep and would divert a large amount of time away from the core application development.

PaaS providers, such as **Heroku** or **PythonAnywhere (used before)**, are therefore the ideal choice for this project. They handle all the server management complexity, allowing me to focus solely on the application code. Heroku, in particular, is well-known for its ease of use and strong support for Python/Django applications, offering a free tier that is perfectly suitable for a project of this scale.

My final decision is to use a **PaaS solution like Heroku** for deployment, as it provides the most efficient and lowest-risk path to getting the application online for demonstration.

5. Conclusion

In this project, I aimed to develop a gamified mobile application to help university students improve their study motivation, task organization, and real-world social interaction. As discussed in the introduction, the app focuses on transforming everyday activities into meaningful achievements through points, badges, and leaderboard systems, encouraging students to stay productive and engaged.

From the market and competitive analysis, I learned valuable lessons from existing

applications such as Habitica, Forest, Headspace, and Duolingo. Each provided insights into effective gamification strategies and potential pitfalls. Based on this analysis, I decided to combine key motivational features—such as progress tracking, fair competition, and a calm interface—while simplifying complex game elements to maintain focus on student productivity and real-life interaction.

In the technology investigation, I selected a decoupled architecture using Django for the backend and Kotlin for the Android frontend to ensure scalability and reliability. The planned migration from SQLite to PostgreSQL will support multi-user performance, and deploying on Heroku allows for efficient testing and demonstration.

Overall, this research confirms both the demand and technical feasibility for a student-focused gamified app. It provides a clear foundation for further development and demonstrates how gamification principles can be effectively applied to support learning and personal growth in higher education.

Reference

Habitica (2020). *Habitica - Gamify Your Life*. [online] Habitica.com. Available at: <https://habitica.com/>.

Forest (2016). *Forest*. [online] Forestapp.cc. Available at: <https://www.forestapp.cc/>.

Headspace (2024). *Meditation and Mindfulness Made Simple - Headspace*. [online] Headspace. Available at: <https://www.headspace.com/>.

Duolingo (2025). *Learn a Language for Free*. [online] Duolingo. Available at: <https://www.duolingo.com/>.